

Grad Cafe SQL Analysis

Nipul Jayasekera (njayase1) · EN.605.256 Modern Software Concepts in Python · Module 3

Eleven questions answered against a PostgreSQL table of 50,004 Grad Cafe admissions results. Every query below was executed through psycopg by `query_data.py`, and this document is generated from that run — the result printed beside each query is the one that query produced, not a figure copied across by hand. Counts are whole numbers; percentages and averages are given to two decimal places.

Question 1

How many entries in the database are from applicants who applied for Fall 2026?

RESULT

Fall 2026 applicant count: 33,208

SQL

```
SELECT COUNT(*) AS fall_2026_entries
FROM applicants
WHERE lower(trim(term)) = 'fall 2026';
```

WHAT IT DOES

Counts every row whose term is Fall 2026. `COUNT(*)` is used rather than `COUNT(term)` because the filter has already guaranteed the column is non-NULL, and `lower(trim(...))` makes the match insensitive to capitalisation and stray whitespace.

Question 2

Among entries that provide a nationality classification, what percentage are international students?

RESULT

Percent international: 47.89%

SQL

```
SELECT ROUND(
    100.0 * COUNT(*) FILTER (WHERE lower(trim(us_or_international)) =
'international')
    / NULLIF(COUNT(*), 0),
    2
) AS percent_international
FROM applicants
WHERE us_or_international IS NOT NULL
AND trim(us_or_international) <> '';
```

WHAT IT DOES

The WHERE clause sets the denominator to entries carrying a usable classification, so missing values are excluded rather than counted as domestic. FILTER supplies the numerator from that same scan. 'American' and 'Other' therefore sit in the denominator but not the numerator, which is what the question asks. NULLIF guards against division by zero on an empty table.

Question 3

What are the average GPA, GRE Quantitative, GRE Verbal and GRE Analytical Writing scores of applicants who provide each metric?

RESULT

Average GPA: 3.77
Average GRE Quantitative: 261.45
Average GRE Verbal: 161.14
Average GRE Analytical Writing: 9.17

SQL

```
SELECT ROUND(AVG(gpa)::numeric, 2) AS avg_gpa,  
       ROUND(AVG(gre)::numeric, 2) AS avg_gre_quant,  
       ROUND(AVG(gre_v)::numeric, 2) AS avg_gre_verbal,  
       ROUND(AVG(gre_aw)::numeric, 2) AS avg_gre_aw  
FROM applicants;
```

WHAT IT DOES

SQL's AVG skips NULLs, so each average is taken over exactly the applicants who reported that one metric. Computing all four in a single statement keeps them independent -- an applicant with a GPA but no GRE still contributes to the GPA average, which is what the question requires. Question 11 examines how much these four figures are moved by values that are out of range for their own scale.

READ BEFORE QUOTING THIS NUMBER

These are the averages of exactly what applicants reported, which is what the question asks for -- but two of them are not usable as test scores. The GRE Quantitative field is bimodal: most values sit in the valid 130-170 band, while a large minority fall between 260 and 340, because those applicants typed their combined GRE total into the box the site labels only 'GRE'. The Analytical Writing average is inflated the same way by placeholder values of 99.99 on a scale that stops at 6. Question 11 counts these and gives the averages restricted to values each scale actually permits; [limitations.pdf](#) discusses what follows from it.

Question 4

What is the average GPA of American applicants who applied for Fall 2026?

RESULT

Average GPA American: 3.79

SQL

```
SELECT ROUND(AVG(gpa)::numeric, 2) AS avg_gpa_american_fall_2026
FROM applicants
WHERE lower(trim(term)) = 'fall 2026'
      AND lower(trim(us_or_international)) = 'american'
      AND gpa IS NOT NULL;
```

WHAT IT DOES

Three conditions applied together: the Fall 2026 term, an American classification, and a reported GPA. The explicit 'gpa IS NOT NULL' is redundant because AVG ignores NULLs anyway, but it states the intent of the question in the query itself.

Question 5

What percentage of Fall 2025 entries are acceptances?

RESULT

Fall 2025 acceptance percentage: 40.74%

SQL

```
SELECT ROUND(
    100.0 * COUNT(*) FILTER (WHERE status ILIKE 'accept%')
    / NULLIF(COUNT(*), 0),
    2
) AS fall_2025_acceptance_percent
FROM applicants
WHERE lower(trim(term)) = 'fall 2025';
```

WHAT IT DOES

The denominator is every Fall 2025 entry, as specified -- including rejections, waitlists and interviews -- while FILTER counts only those whose cleaned status begins with 'Accept'. One pass over the table produces both halves of the fraction.

Question 6

What is the average GPA of accepted applicants who applied for Fall 2026?

RESULT

Average GPA Acceptance: 3.78

SQL

```
SELECT ROUND(AVG(gpa)::numeric, 2) AS avg_gpa_accepted_fall_2026
FROM applicants
WHERE lower(trim(term)) = 'fall 2026'
      AND status ILIKE 'accept%'
      AND gpa IS NOT NULL;
```

WHAT IT DOES

The same shape as Question 4, with the acceptance status swapped in for the nationality filter. Comparing the two answers against Question 3's overall GPA shows how little the reported GPA separates accepted applicants from the field as a whole.

Question 7

How many entries are from applicants who applied to Johns Hopkins University for a master's degree in Computer Science?

RESULT

JHU Masters Computer Science count: 18

SQL

```
SELECT COUNT(*) AS jhu_masters_cs_entries
FROM applicants
WHERE (program ILIKE '%johns hopkins%' OR program ~* '\mjhu\M')
      AND program ILIKE '%computer science%'
      AND (degree ILIKE 'master%'
           OR lower(trim(degree)) IN ('ms', 'm.s.', 'msc', 'm.sc.', 'meng', 'm.eng.'));
```

WHAT IT DOES

Uses the original downloaded fields, as the question requires. 'program' holds '<program>, <university>', so one ILIKE matches the university half and another the department half. 'JHU' is matched with the word boundaries \m and \M rather than '%jhu%', which would also fire inside unrelated words. The degree test accepts the cleaned 'Masters' label plus the common abbreviations, but not MFA, MBA or JD, which are different credentials.

Question 8

How many Fall 2026 entries are acceptances from applicants applying for a PhD in Computer Science at Georgetown, MIT, Stanford or Carnegie Mellon? (original downloaded fields)

RESULT

Original-field count: 30

SQL

```
SELECT COUNT(*) AS cs_phd_acceptances_fall_2026
FROM applicants
WHERE lower(trim(term)) = 'fall 2026'
  AND status ILIKE 'accept%'
  AND degree ~* '^ph\.\s*d'
  AND program ILIKE '%computer science%'
  AND (
    program ILIKE '%georgetown%'
    OR program ILIKE '%massachusetts institute of technology%'
    OR program ~* '\mmit\M'
    OR program ILIKE '%stanford%'
    OR program ILIKE '%carnegie mellon%'
    OR program ~* '\mcmu\M'
  );
```

WHAT IT DOES

All five restrictions are applied at once in a single WHERE clause: term, status, degree, department and university. The university test is the bracketed OR group, so it narrows the row set rather than widening it -- without the parentheses the OR would defeat every AND above it and return far too many rows.

Question 9

Repeat Question 8 using the LLM-generated program and university fields, and compare the two counts.

RESULT

Original-field count: 30
LLM-field count: 30
Difference: +0

SQL

```
SELECT COUNT(*) AS cs_phd_acceptances_fall_2026_llm
FROM applicants
WHERE lower(trim(term)) = 'fall 2026'
  AND status ILIKE 'accept%'
  AND degree ~* '^ph\.\s*d'
  AND llm_generated_program ILIKE '%computer science%'
  AND (
    llm_generated_university ILIKE '%georgetown%'
    OR llm_generated_university ILIKE '%massachusetts institute of technology%'
    OR llm_generated_university ~* '\mmit\M'
    OR llm_generated_university ILIKE '%stanford%'
    OR llm_generated_university ILIKE '%carnegie mellon%'
    OR llm_generated_university ~* '\mcmu\M'
  );
```

WHAT IT DOES

Identical to Question 8 except that the department and university come from the standardized columns, while term, degree and status still come from the original downloaded fields as instructed.

READ BEFORE QUOTING THIS NUMBER

The two counts are equal here, and not by coincidence: they select the identical set of rows, not merely the same number of them. Grad Cafe stores each result's school as a foreign key into its own controlled vocabulary rather than as applicant free text, so these four well-known universities already arrive with one full, correctly spelled name ('Massachusetts Institute of Technology (MIT)', 'Stanford University'). Standardizing a name that is already canonical cannot change which rows match. The standardizer is not idle overall -- it rewrites the university on 4,768 of the 50,000 rows, folding 'University of Wisconsin - Madison' and 'University of California (UCLA)' onto single canonical spellings -- but none of that work falls inside this particular filter. A difference would appear for schools the site stores inconsistently, or for a department name like 'CS' that only the LLM column spells out.

Question 10 — my own question

Of the ten universities with the most Fall 2026 entries, which reports the highest acceptance rate, and how wide is the spread?

RESULT

Highest acceptance rate among the ten busiest Fall 2026 universities: University of Toronto at 39.70% (212 of 534 entries)
Lowest: Yale University at 14.22% (90 of 633 entries)

University	Entries	Acceptances	Acceptance rate
University of Toronto	534	212	39.70%
University of Chicago	579	157	27.12%
University of Washington	623	161	25.84%
University of Michigan, Ann Arbor	584	150	25.68%
University of California, Berkeley	722	184	25.48%
Princeton University	597	123	20.60%
University of Wisconsin–Madison	604	121	20.03%
Massachusetts Institute of Technology	574	107	18.64%
Stanford University	770	142	18.44%
Yale University	633	90	14.22%

SQL

```
SELECT university,
       entries,
       acceptances,
       ROUND(100.0 * acceptances / NULLIF(entries, 0), 2) AS acceptance_percent
FROM (
  SELECT llm_generated_university AS university,
         COUNT(*) AS entries,
         COUNT(*) FILTER (WHERE status ILIKE 'accept%') AS acceptances
  FROM applicants
  WHERE lower(trim(term)) = 'fall 2026'
        AND llm_generated_university IS NOT NULL
  GROUP BY llm_generated_university
  ORDER BY entries DESC
  LIMIT 10
) AS busiest
ORDER BY acceptance_percent DESC;
```

WHAT IT DOES

The inner query groups Fall 2026 entries by standardized university and keeps the ten with the most submissions, so every rate is drawn from a reasonable sample rather than from a school with three entries. The outer query turns each pair of counts into a percentage and ranks by it. Grouping on the standardized column rather than the raw one matters here: the raw field spells the same school several ways, which would split one university across several groups and push all of them out of the top ten.

Question 11 — my own question

How many self-reported metrics are impossible for their own scale, and how far do they move the averages reported in Question 3?

RESULT

Impossible values reported across the four metrics: 2,715
Largest distortion: GRE Quantitative (130-170), whose headline average is 95.71 too high

Metric	Reported	Out of range	Average (all)	Average (in range)	Distortion
GPA (0-4.0)	29,482	231	3.77	3.75	+0.02
GRE Analytical Writing (0-6)	2,938	153	9.17	4.33	+4.84
GRE Verbal (130-170)	3,139	15	161.14	160.37	+0.77
GRE Quantitative (130-170)	3,769	2,316	261.45	165.74	+95.71

SQL

```
SELECT 'GPA (0-4.0)' AS metric,
       COUNT(gpa) AS reported,
       COUNT(*) FILTER (WHERE gpa > 4.0) AS out_of_range,
       ROUND(AVG(gpa)::numeric, 2) AS average_all,
       ROUND(AVG(gpa) FILTER (WHERE gpa <= 4.0)::numeric, 2) AS average_in_range
FROM applicants
UNION ALL
SELECT 'GRE Quantitative (130-170)',
       COUNT(gre),
       COUNT(*) FILTER (WHERE gre < 130 OR gre > 170),
       ROUND(AVG(gre)::numeric, 2),
       ROUND(AVG(gre) FILTER (WHERE gre BETWEEN 130 AND 170)::numeric, 2)
FROM applicants
UNION ALL
SELECT 'GRE Verbal (130-170)',
       COUNT(gre_v),
       COUNT(*) FILTER (WHERE gre_v < 130 OR gre_v > 170),
       ROUND(AVG(gre_v)::numeric, 2),
       ROUND(AVG(gre_v) FILTER (WHERE gre_v BETWEEN 130 AND 170)::numeric, 2)
FROM applicants
UNION ALL
SELECT 'GRE Analytical Writing (0-6)',
       COUNT(gre_aw),
       COUNT(*) FILTER (WHERE gre_aw < 0 OR gre_aw > 6),
       ROUND(AVG(gre_aw)::numeric, 2),
       ROUND(AVG(gre_aw) FILTER (WHERE gre_aw BETWEEN 0 AND 6)::numeric, 2)
FROM applicants;
```

WHAT IT DOES

Grad Cafe validates nothing an applicant types, so the table holds GRE Analytical Writing scores of 99.99 and GPAs on a 10-point scale. Each branch of the UNION counts how many values a metric has, how many of those are outside the range the scale allows, and the average with and without them. The final column is the gap between those two averages -- that is, the amount of Question 3's answer that is an artefact of unvalidated input rather than a fact about applicants. UNION ALL rather than UNION so that two metrics with identical counts are not silently collapsed into one row.